



UNIVERSITÀ DI PARMA

SIFT: Scale-Invariant Feature Transform



- From features to SIFT
- Stage 1 – Scale-Space Construction
- Stage 2 – Keypoint Detection
- Stage 3 – Keypoint Localization
- *Stage 4 – Orientation Assignment*
- *Stage 5 – Keypoint Descriptor*
- Invariance properties and practical notes



- D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints", IJCV 2004
 - The definitive SIFT paper
- D. G. Lowe, "Object Recognition from Local Scale-Invariant Features", ICCV 1999
 - Original SIFT conference paper
- R. Szeliski. Computer Vision: Algorithms and Applications, 2010
- CS231A – Computer Vision, Prof. S. Savarese, Stanford University
- K. Mikolajczyk, C. Schmid, "A Performance Evaluation of Local Descriptors", TPAMI 2005

- Previous lecture: feature detection
 - Harris corner detector – not invariant to scale
 - Scale-invariant concept: search over window sizes / pyramid
 - SIFT mentioned as a complete scale-invariant solution
- This lecture: SIFT in depth
- SIFT provides a full pipeline
 - Detection: where are the keypoints?
 - Description: what do they look like? Next lecture!
- Goal: detect keypoints that are robust to scale, rotation, and illumination changes

- Proposed by David Lowe (ICCV 1999, IJCV 2004)
- Four computational stages
 - 1. Scale-Space Extrema Detection
 - 2. Keypoint Localization
 - 3. Orientation Assignment
- Invariant to scale and rotation; partially invariant to illumination



UNIVERSITÀ DI PARMA

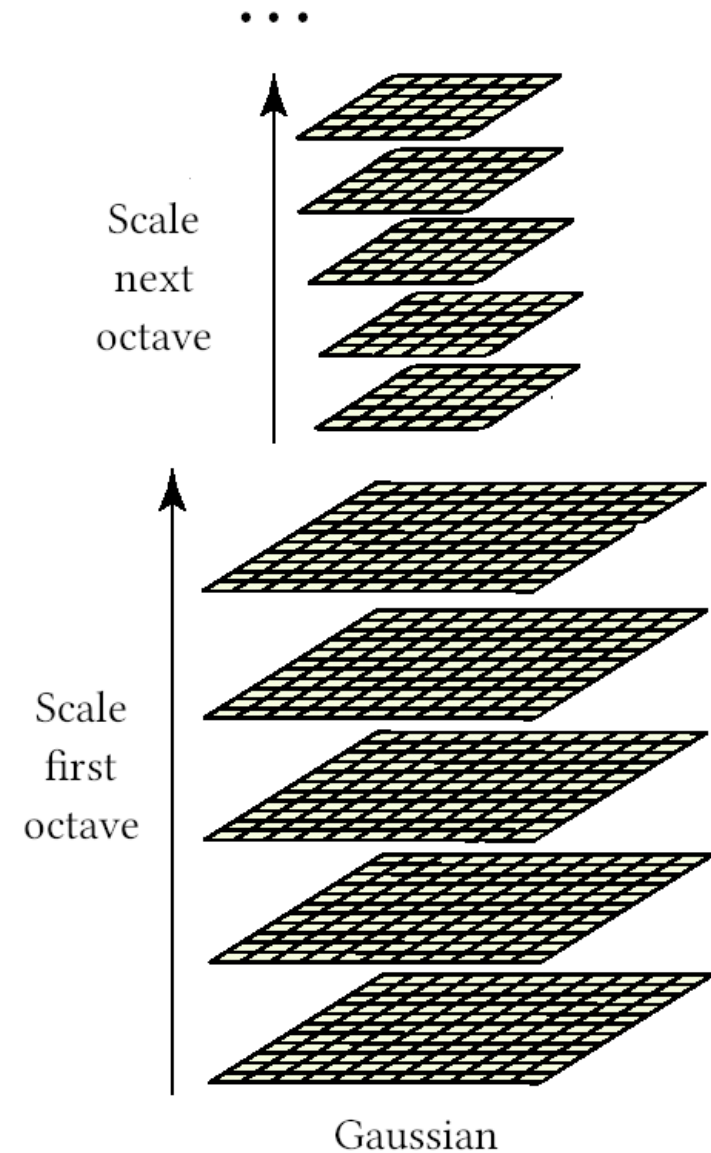
Stage 1: Scale-Space Construction

- Goal: represent image content at multiple scales simultaneously
- Scale space: $L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$
 - G: 2D Gaussian with standard deviation σ
 - Why gaussian?
 - Lindeberg (1994): Gaussian is the only kernel that does not introduce new extrema
 - Theoretically justified choice for scale-space construction
- SIFT builds a multi-scale, multi-resolution pyramid
 - In order to build a pyramid we can independently act on (W,H) or on σ
 - (W,H) $\rightarrow$ directly modify scale
 - σ $\rightarrow$ more blurring or sharp image (coarse/fine scale)

- We use octaves for $L(x, y, \sigma)$. Usual initial values
 - For each octave we have $s = 3$ levels
 - Scale factor for levels $\rightarrow k = 2^{1/s} \approx 1.26$
 - $\sigma_0 = 1.6$
 - $\sigma_i = \sigma_0 \times k^i$
- Octave 0, σ values: 1.60, 2.02, 2.54, 3.20, 4.03, 5.08 (s+3 levels, why?)
- Octave 1:
 - Downsampling by 2 starting from intermediate level of Octave 0 ($\sigma = 3.20$)
 - σ values: same as before, but on a different scale, namely we have to multiply by 2
 - σ actual values: 3.20, 4.03, 5.08, 6.40, 8.06, 10.16
- Ad so on...

Gaussian Scale Space

- Resulting Scale Space



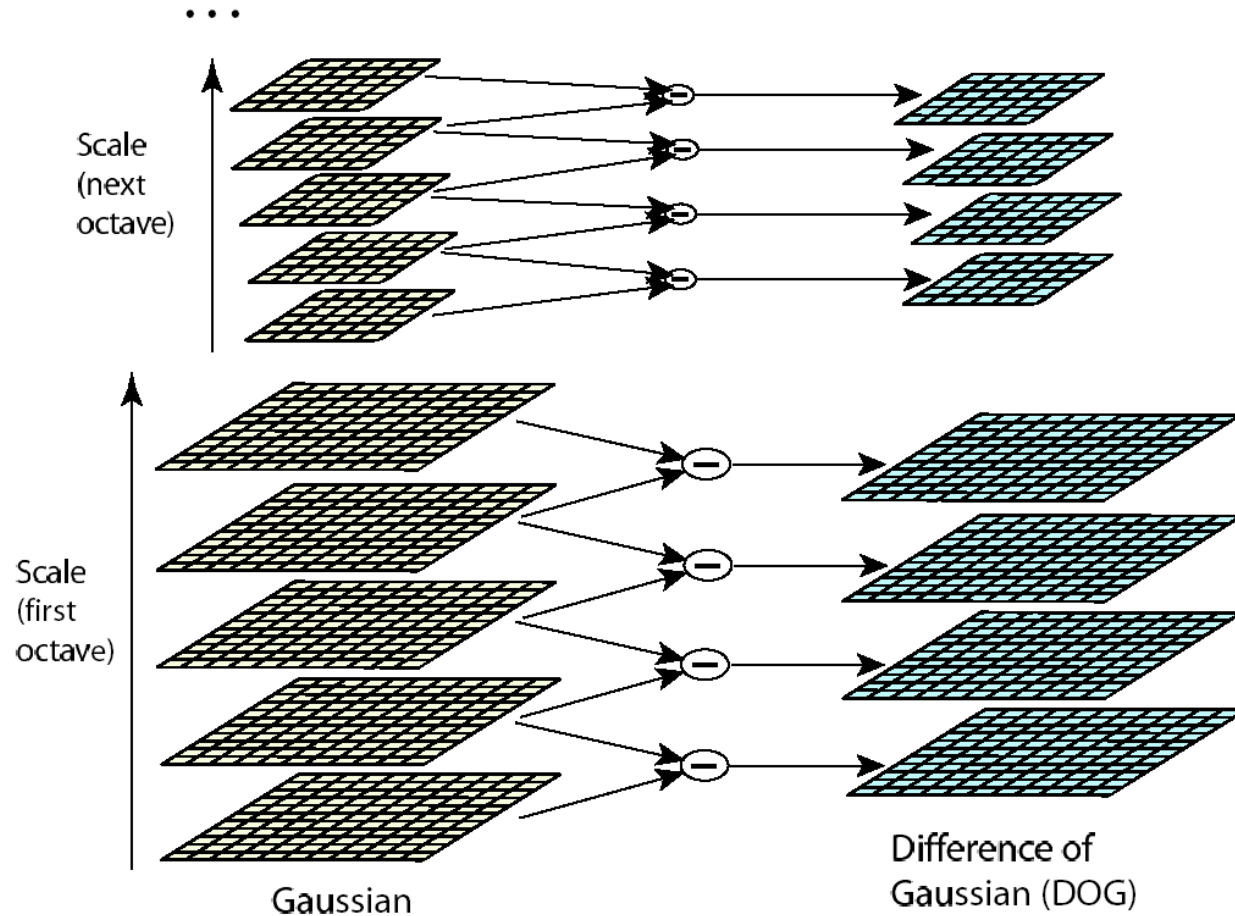
- For the Octave o at level i we have: $\sigma_{actual} = \sigma_0 \times k^i \times 2^o$
- How many octaves?
 - This can be computed from the original image size, usually
 - $N = \lfloor \log_2(\min(W,H)) \rfloor - 3$
 - Not always used, as example OpenCV has a default number (4 or 5)
- Large σ should imply large kernels $\rightarrow$ optimization trick:
 - $G(\sigma_2) = G(\sqrt{(\sigma_2^2 - \sigma_1^2)}) * G(\sigma_1)$

- Laplacian of Gaussian
 - $LoG(x, y, \sigma) = \nabla^2(G(x, y, \sigma) * I(x, y))$
 - Gaussian filtering $\rightarrow$ remove noise
 - Second derivative $\rightarrow$ relevant info (maybe)
 - Blobs
- Optimized as:
 - $LoG(x, y, \sigma) = \nabla^2(G(x, y, \sigma)) * I(x, y)$

- Blob detector: find local extrema of LoG
 - Responds maximally to blobs of a size matching σ
- Scale-normalized LoG: $\sigma^2 \cdot \nabla^2 G$
 - Without σ^2 normalization, response decreases with scale
 - Normalization needed for true scale invariance
- Extremum of $\sigma^2 \cdot \nabla^2 G$ over $(x, y, \sigma) \rightarrow$ blob at position and scale
- Problem: LoG is expensive to compute directly

- DoG $\approx$ LoG (efficient approximation)
- $D(x, y, \sigma_i) = L(x, y, \sigma_{i+1}) - L(x, y, \sigma_i)$
 - DoG is the difference of two adjacent blurred images
- Relationship to LoG:
 - $D \approx (k - 1) \sigma^2 \nabla^2 G$
 - $(k - 1)$ is a constant factor, does not affect extremum location
- Key advantages:
 - DoG images are computed as a by-product of building the pyramid
 - No extra filtering required

Difference of Gaussians (DoG)



- Assuming $s + 3$ Gaussian images for each octave
- DoG images: adjacent Gaussian differences
 - $s + 2$ DoG images per octave
 - Extrema searched in the s inner DoG images
- Total keypoint search space: $s \text{ levels} \times (\text{octaves}) \times (\text{image pixels})$

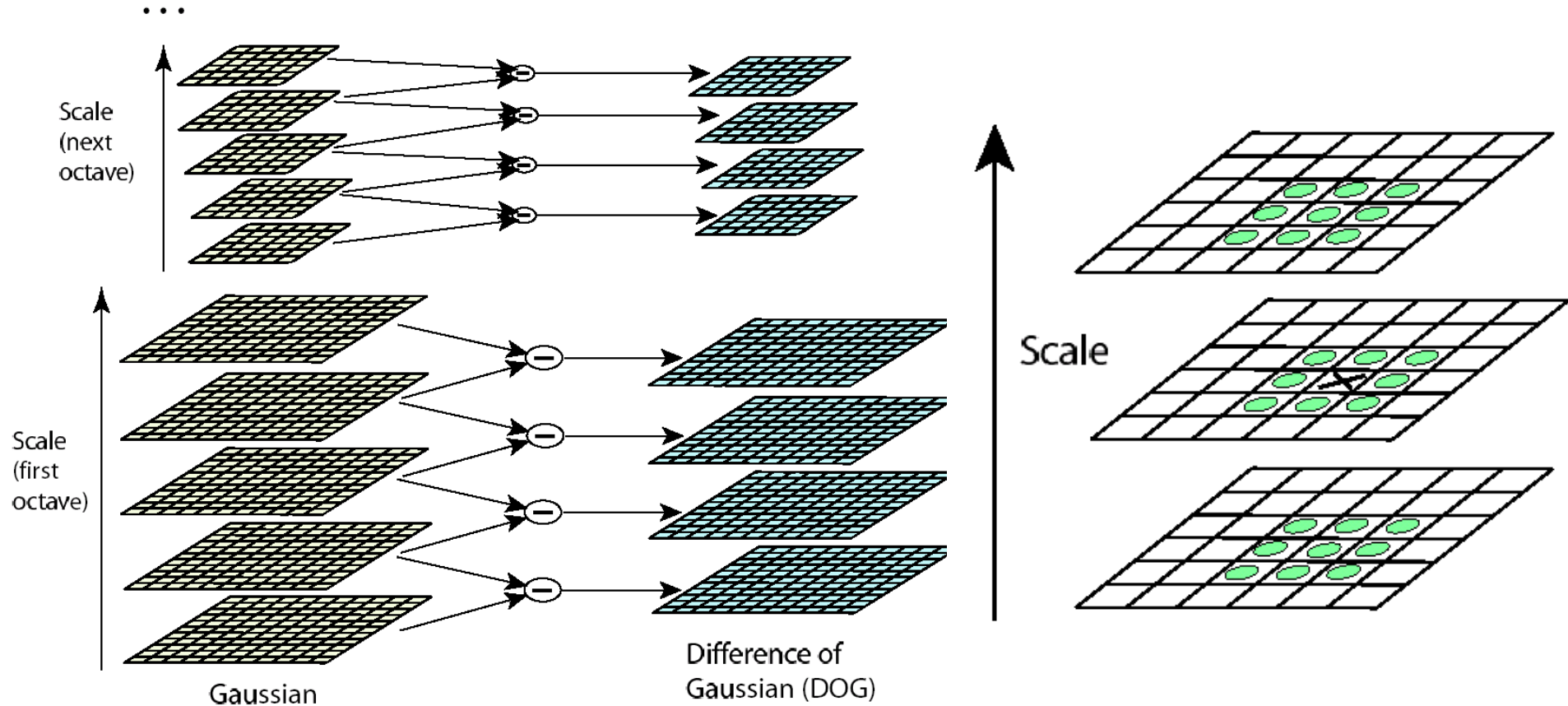


UNIVERSITÀ DI PARMA

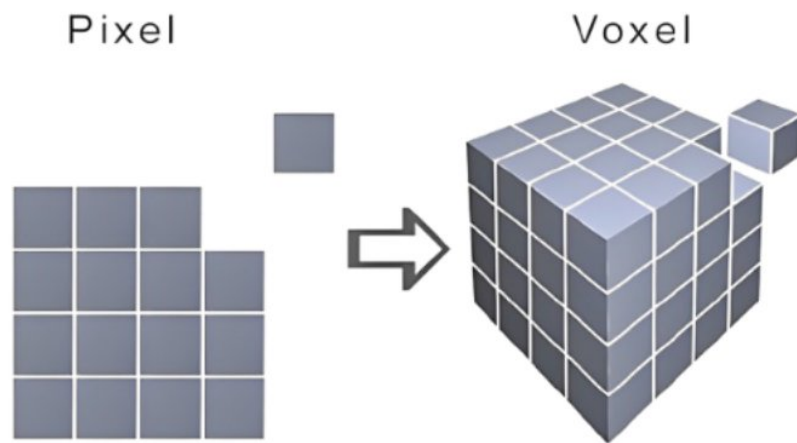
Stage 2: Keypoint Detection

- A keypoint candidate is a local extremum of $D(x, y, \sigma_i)$
- Each sample compared to 26 neighbors
 - 8 neighbors at the same scale
 - 9 neighbors at the scale above
 - 9 neighbors at the scale below
- Selected only if larger (or smaller) than ALL 26 neighbors
- This detects scale-space blobs — local maxima and minima
- Cost
 - 3/4 of all points are eliminated after first neighbor check
 - Inexpensive: DoG is already computed

Scale-Space Extrema Detection



- Resulting candidates actually feature 3 coordinates
 - x , y , and σ
- They are “**voxels**”
- The same as pixel but in a 3D grid





UNIVERSITÀ DI PARMA

Stage 3: Keypoint Localization

- Candidate keypoints are at discrete (pixel, scale) locations (x, y, σ)
 - SIFT goes beyond to produce sub-pixel/sub-scale interpolation
 - From (x, y, σ) to $(x + \Delta x, y + \Delta y, \sigma + \Delta \sigma)$
- $D(x, y, \sigma)$ is approximated using Taylor expansion up to the second order
 - Assuming $\mathbf{x} = (\Delta x, \Delta y, \Delta \sigma)^T$, offset from the candidate we can approximate $D()$ as:
 - $D \approx \hat{D}(\mathbf{x}) = D + (\partial D^T / \partial \mathbf{x}) \mathbf{x} + (1/2) \mathbf{x}^T (\partial^2 D / \partial \mathbf{x}^2) \mathbf{x}$
- To find minima or maxima find where the derivative is 0
 - Sub-voxel offset: $\hat{\mathbf{x}} = -(\partial^2 D / \partial \mathbf{x}^2)^{-1} (\partial D / \partial \mathbf{x})$
- The $\hat{\mathbf{x}}$ is the offset to be used to refine the candidate keypoints

- When the offset is too large we can refine the result
- If $|\hat{x}| > 0.5$ in any dimension:
 - Move the keypoint candidate to the adjacent sample
 - Repeat; typically converges in 1–3 iterations
- Improves localization accuracy
 - reduces sensitivity to sampling

- After Taylor fit, evaluate D at the sub-voxel location:
- $D(\hat{x}) = D + (1/2)(\partial D / \partial x)^T \hat{x}$
- Threshold check: $|D(\hat{x})| < 0.03 \rightarrow \text{discard}$
 - Value 0.03 is Lowe's threshold for images normalized to $[0, 1]$
- These points lie near zero-crossings of DoG
- They are sensitive to noise and cannot be matched reliably
- Discarding them significantly reduces the number of keypoints

- DoG has strong responses along edges: poor localization across the edge
- 2×2 Hessian of D in image space:
 - $H = [[D_{xx}, D_{xy}], [D_{xy}, D_{yy}]]$
 - Eigenvalues α, β represent principal curvatures of D
- Ratio of curvatures:
 - $\text{Tr}(H)^2 / \text{Det}(H) = (\alpha + \beta)^2 / (\alpha\beta)$
 - For a corner: $\alpha \approx \beta \rightarrow$ ratio close to 4
 - For an edge: $\alpha \gg \beta \rightarrow$ ratio very large
- Threshold: discard if $\text{Tr}(H)^2 / \text{Det}(H) > (r + 1)^2 / r$
 - Lowe uses $r = 10 \rightarrow$ threshold = 12.1
 - Does not require computing actual eigenvalues

- For each DoG extremum
 - 1. Taylor expansion $\rightarrow$ sub-voxel offset $\hat{x}$
 - 2. If $|\hat{x}| > 0.5$: move to adjacent sample and repeat
 - 3. If $|D(\hat{x})| < 0.03$: discard (low contrast)
 - 4. If $\text{Tr}(H)^2/\text{Det}(H) > 12.1$: discard (edge)
- Result: stable, well-localized keypoints with assigned (x, y, σ)
- Reduction in keypoints:
 - Typically $\sim 80\text{--}90\%$ of initial extrema are discarded
 - Remaining keypoints are stable and distinctive



UNIVERSITÀ DI PARMA

SIFT: Scale-Invariant Feature Transform

Question time!